

AI Health Companion

Complete Product & Engineering Documentation

Target platforms: iOS + Web

Recommended iOS: Swift + SwiftUI

Recommended Web: Next.js + TypeScript

Backend: NestJS + PostgreSQL + pgvector + Redis

Document scope: Product, SRS, UX, architecture, AI, safety, database, API, security, testing, operations, roadmap, and development workflow.

Positioning: an AI-powered health companion that helps users understand health information, track health data, follow clinician-provided instructions, and navigate care. It is not positioned as a replacement for a doctor or emergency service.

1. Executive Summary

The product is a privacy-focused AI Health Companion available on iOS and the web. The iOS application is the primary personal health experience and integrates with Apple Health/HealthKit. The web application provides a full account dashboard, AI chat, document/report management, health timeline, and account controls. Both clients use the same backend, health memory, AI gateway, and data layer.

The product combines conversational AI, structured health memory, medical-document understanding, image analysis for supported low-risk use cases, voice interaction, health tracking, reminders, and care discovery. The core engineering principle is that medical safety is implemented as a system—not as a single prompt.

Core value proposition

- Understand health information in plain language.
- Keep a longitudinal, user-controlled health timeline.
- Turn clinician-provided instructions into reminders and trackable tasks.
- Connect relevant wearable/HealthKit data to trends.
- Use multimodal AI for supported documents, images, and voice.
- Recognize potentially urgent situations and escalate instead of continuing casual conversation.
- Help users prepare for and navigate professional care.

Primary platforms

Platform	Technology	Primary role
iOS	Swift + SwiftUI	Primary health companion; HealthKit, camera, microphone, push notifications
Web	Next.js + TypeScript	Dashboard, chat, documents, timeline, account/privacy
Backend	NestJS + TypeScript	Shared API and business logic
Database	PostgreSQL + pgvector	Structured health data + semantic memory
Storage	AWS S3 or equivalent	Reports, images, audio and generated artifacts
Queues	Redis + BullMQ	Document processing, reminders, background jobs

2. Product Scope

2.1 User capabilities

- Create an account and establish a health profile.
- Chat with the AI health assistant by text.
- Speak to the assistant using voice.
- Upload PDFs and medical reports.
- Upload supported health images such as minor injuries, rashes, swelling, or wounds.
- Track symptoms, pain, sleep, water, medication adherence, and other user-selected metrics.
- Connect Apple Health data through HealthKit on iOS.
- Maintain a structured health memory with provenance and user controls.
- Store clinician-provided instructions and convert them into tasks/reminders.
- View a chronological health timeline.
- Search for nearby or online care options.
- Export, correct, or delete supported health information.

2.2 Explicit non-goals

- Do not market the system as a replacement for doctors.
- Do not allow the AI to independently prescribe medication.
- Do not allow the AI to change a prescription or dosage on its own.
- Do not claim definitive diagnosis from symptoms or an image.
- Do not use one wearable measurement as proof of a medical condition.
- Do not provide emergency response as a substitute for local emergency services.

3. Product Requirements Document (PRD)

3.1 Target user

Adults who want a single place to ask health questions, understand health records, track day-to-day health information, remember clinician instructions, and prepare for professional care. The initial product should focus on clarity, safety, convenience, and continuity rather than attempting to cover every clinical workflow.

3.2 Core user journeys

- Question → AI asks relevant follow-up questions → safety check → explanation → self-care guidance or care recommendation.
- Upload report → secure processing → structured extraction → plain-language explanation → timeline entry → optional AI follow-up.
- Tell AI a doctor instruction → verify exact instruction → create treatment task → reminder → completion tracking → progress view.
- Connect HealthKit → permission screen → import approved data types → normalize measurements → trend display → AI contextual summary.
- Upload image → image-quality check → supported-use check → cautious analysis → uncertainty + warning signs → care recommendation if needed.
- Voice question → speech-to-text → safety/triage → reasoning → spoken answer → transcript saved to conversation.

3.3 MVP

- iOS authentication and onboarding.
- Web authentication and account dashboard.
- AI text chat.
- Health profile.
- Structured health memory.
- Safety/triage layer.
- Medical PDF/image upload and basic extraction.
- Symptom tracking.
- Health timeline.
- Clinician instruction → task/reminder workflow.
- Push notifications on iOS.
- Privacy, consent, audit, export/delete foundation.

3.4 Post-MVP

- Voice conversations.

- Health image analysis.
- HealthKit integrations.
- Health trends and longitudinal summaries.
- Doctor/care discovery.
- Online consultation navigation.
- Apple Watch workflows where appropriate.
- Clinician-facing workflows and shared reports.
- Family/caregiver features with explicit consent.

4. Software Requirements Specification (SRS)

4.1 Functional requirements

ID	Requirement	Acceptance direction
FR-001	Authentication	Secure sign-up/login, session management, recovery, logout.
FR-002	Profile	Store user-controlled health profile and preferences.
FR-003	AI chat	Conversation with relevant context and persistent history.
FR-004	Safety triage	Every health interaction passes through safety logic.
FR-005	Documents	Securely upload supported PDFs/images and process asynchronously.
FR-006	Extraction	Extract structured fields with source/provenance.
FR-007	Images	Run supported visual analysis with quality and uncertainty checks.
FR-008	Memory	Create/update memories with source, status, confidence and timestamps.
FR-009	Timeline	Show chronological health events and measurements.
FR-010	Treatment plans	Represent clinician/user-provided instructions as structured tasks.
FR-011	Reminders	Schedule and deliver reminders; track completion.
FR-012	Voice	STT → safety → reasoning → TTS pipeline.
FR-013	HealthKit	Read only user-approved HealthKit types; normalize securely.
FR-014	Care discovery	Search/filter care options and provide appropriate escalation.
FR-015	Data controls	Correct, export and delete supported data.
FR-016	Auditability	Record security-sensitive/admin actions without unnecessary health content.

4.2 Non-functional requirements

- Security: encryption in transit and at rest, least privilege, strong authorization, secrets management.
- Privacy: data minimization, explicit consent, retention policies, export/delete, regional controls.
- Reliability: target 99.9% availability for core APIs once production maturity is reached.
- Performance: normal APIs should generally respond quickly; AI/document processing may be asynchronous.
- Scalability: architecture should support growth from early MVP to large user populations without redesigning the data model.
- Accessibility: Dynamic Type, VoiceOver, readable contrast, keyboard navigation on web, transcripts for voice.
- Observability: logs, metrics, traces, AI latency/cost, safety events, queue health and integration health.

5. iOS Architecture

5.1 Native iOS stack

Area	Choice
Language	Swift
UI	SwiftUI
Architecture	MVVM or feature-based unidirectional state architecture
Networking	URLSession with typed API layer
Local storage	SwiftData/Core Data as appropriate for local cache; Keychain for secrets
Health	HealthKit
Notifications	APNs / UserNotifications
Voice	Apple speech/audio APIs or approved provider integration
Camera	AVFoundation / Photos
Analytics	Privacy-minimized product analytics
Crash reporting	Sentry or equivalent

5.2 iOS modules

- Authentication
- Onboarding and consent
- Home dashboard
- AI chat
- Voice assistant
- Health profile
- Health timeline
- Documents
- Image analysis
- Plans/tasks
- Reminders
- HealthKit integration
- Care discovery
- Settings/privacy

5.3 HealthKit strategy

The iOS app should request only the HealthKit data types that provide a clear product benefit. Permissions must be understandable and granular. The backend should receive normalized data needed for the product rather than attempting to mirror every available HealthKit field.

- Start with high-value metrics such as sleep, steps/activity, heart rate, resting heart rate, weight and workouts where supported and appropriate.
- Treat HealthKit data as measurements, not diagnoses.
- Store source, timestamp, unit, device/source where available, and ingestion status.
- Allow the user to disconnect HealthKit and explain what happens to previously imported data.
- Do not request sensitive HealthKit permissions without a concrete feature requiring them.

6. Web Architecture

6.1 Next.js application

- Next.js + TypeScript.
- Responsive dashboard optimized for desktop and tablet.
- Server-side rendering where useful; authenticated application routes behind authorization.
- Shared API client with generated types.
- Accessible components and keyboard navigation.
- Secure upload flow using short-lived signed URLs rather than routing large files through the web server unnecessarily.

6.2 Web screens

Screen	Main functions
Landing	Product explanation, safety/privacy positioning, signup/login
Dashboard	Health snapshot, recent timeline events, tasks, AI entry point
AI Chat	Conversation, attachments, voice where supported
Health	Profile, conditions, allergies, medications, measurements
Timeline	Chronological events, filters, sources
Reports	Uploaded reports, extracted data, explanations
Plans	Clinician instructions, tasks, reminders, completion
Trends	Graphs and longitudinal summaries
Care	Provider/specialty/location discovery
Settings	Privacy, consent, integrations, export/delete

7. Shared Backend Architecture



7.1 Recommended backend stack

Component	Recommendation
API	NestJS + TypeScript
Database	PostgreSQL
Vector search	pgvector
Cache	Redis
Background jobs	BullMQ
Storage	AWS S3
Authentication	Managed identity provider or hardened auth service
Infrastructure	AWS or equivalent
CI/CD	GitHub Actions
Monitoring	Sentry + cloud logging/metrics/tracing

8. AI / Medical Intelligence Architecture

8.1 AI request pipeline

```
User input
↓
Input validation + abuse controls
↓
Safety / triage classifier
↓
Relevant health-context retrieval
↓
AI orchestrator
  ■■■ LLM reasoning
  ■■■ Vision model
  ■■■ Document extraction
  ■■■ Speech services
↓
Response safety validation
↓
Response + structured actions
  ■■■ answer
  ■■■ follow-up questions
  ■■■ warning signs
  ■■■ care recommendation
  ■■■ optional task/reminder action
```

8.2 AI responsibilities

- Explain health information in plain language.
- Ask clarifying questions when information is incomplete.
- Summarize relevant history instead of dumping all history into the prompt.
- Identify when professional evaluation is appropriate.
- Convert explicitly stated clinician instructions into structured tasks.
- Summarize longitudinal trends without overclaiming causality.
- Explain documents and images with uncertainty.

8.3 AI must not

- Present an uncertain inference as a confirmed diagnosis.
- Invent medical history, test results, medications or doctor instructions.

- Change a medication dose independently.
- Tell a user to ignore a potentially serious symptom.
- Treat an uploaded document's instructions as system/developer instructions.
- Use private health information from one user in another user's context.

8.4 Health memory model

```
{
  "fact": "User reported recurring headaches",
  "source": "user_conversation",
  "source_id": "message_123",
  "date": "2026-09-20",
  "confidence": 0.91,
  "status": "user_reported"
}
```

Suggested status values: user_reported, document_extracted, wearable, clinician_provided, ai_inferred, user_confirmed, superseded. AI-inferred facts should not silently become confirmed medical history.

9. Medical Safety & Triage

9.1 Independent safety layer

Safety should be implemented outside the conversational prompt. A combination of deterministic rules, safety classification, structured symptom extraction, escalation thresholds, and response validation should determine whether ordinary conversation is appropriate.

9.2 Example escalation categories

Level	Example behavior
Emergency	Potentially life-threatening pattern → advise immediate local emergency care; do not continue routine self-care flow.
Urgent	Prompt professional evaluation, same-day/soon depending on scenario and local context.
Routine	Provide general information, self-care where appropriate, and routine professional follow-up when warranted.
Informational	General education with normal uncertainty language.

9.3 Safety test cases

- Chest pain + shortness of breath → emergency escalation rather than casual advice.
- Sudden neurological symptoms → emergency escalation.
- Medication change request → do not independently alter prescription.
- Unclear image → state that image quality is insufficient for reliable assessment.
- Malicious text embedded in a PDF → treat it as document data, not instructions.
- Conflicting medical-history facts → surface conflict and ask user to confirm.
- Missing critical symptom detail → ask targeted follow-up questions before giving a non-urgent interpretation.

10. Medical Documents & Image Processing

10.1 Document pipeline

```
Upload
↓
MIME/type validation + size limits
↓
Malware/security scan
↓
```



```
Object storage
↓
OCR / document parser
↓
Structured extraction
↓
Validation / confidence checks
↓
Store source-linked facts
↓
AI explanation
↓
Timeline / memory only when appropriate
```

- Support PDF, JPG, PNG and HEIC where practical.
- Keep original file and extracted representation separately.
- Every extracted medical fact should retain a source reference to the document/page when possible.
- Do not overwrite original documents with AI-generated summaries.
- Allow users to correct extracted fields.

10.2 Image analysis

- Perform image-quality and supported-use checks first.
- Do not claim certainty from appearance alone.
- Provide warning signs and professional-care guidance when appropriate.
- Do not diagnose from an image as a definitive medical conclusion.
- Keep image retention configurable and clearly disclosed.

11. Voice Experience

Voice should feel conversational but must use the same safety and health-context pipeline as text.

```
Microphone
↓
Speech-to-text
↓
Safety / triage
↓
Context retrieval
↓
AI response
↓
Text-to-speech
↓
Audio playback + transcript
```

- Allow users to stop generation immediately.
- Show a transcript for accessibility and review.
- Do not hide urgent-care messaging inside audio only.
- Store audio only if the user has been clearly informed and the feature requires it.

12. Health Tracking & Timeline

12.1 Daily tracking

- How are you feeling?
- Symptom severity
- Pain score

- Sleep
- Water
- Medication/task completion
- User-selected measurements

12.2 Timeline event model

```
TimelineEvent
- id
- user_id
- event_type
- occurred_at
- source_type
- source_id
- payload
- confidence
- created_at
- updated_at
```

The timeline should distinguish user-entered facts, device measurements, document-extracted facts, clinician-provided information and AI-generated summaries.

13. Treatment Plans, Tasks & Reminders

A user may tell the assistant: “My doctor said I should drink five glasses of water every day” or “Take this medicine after dinner for ten days.” The system should first preserve the instruction as stated, then ask for confirmation if required before creating a structured task.

Object	Purpose
DoctorInstruction	Original instruction and source/context
TreatmentPlan	Grouped plan with start/end dates and status
Task	Action the user must complete
Reminder	Notification schedule
Completion	Actual completion record

- Never invent dosage, timing, duration or frequency.
- Allow users to edit or cancel reminders without rewriting the original clinician instruction.
- Keep completion tracking separate from the medical instruction itself.

14. Database Architecture

14.1 Core tables

Table	Purpose
users	Identity/account
profiles	User profile and preferences
health_conditions	Known/user-reported conditions
allergies	Allergy records
medications	Medication records
doctor_instructions	Source instructions
treatment_plans	Structured plans
tasks	Trackable actions

Table	Purpose
task_completions	Completion history
reminders	Notification schedules
conversations	Chat sessions
messages	Conversation messages
health_memories	Persistent structured memories
symptoms	Symptom definitions/records
symptom_events	Observed symptom events
health_measurements	Normalized measurements
medical_documents	Original document metadata
document_extractions	Extracted fields
health_images	Uploaded health images
image_analysis	Vision results
healthkit_connections	Integration metadata
wearable_measurements	Imported device measurements
timeline_events	Unified timeline
appointments	Care navigation/appointments
care_providers	Provider directory records
consents	Consent records
audit_logs	Security/admin audit trail

14.2 Vector memory

Use pgvector for semantic retrieval of relevant health memories and prior conversation summaries. Retrieval must always be scoped by user identity and authorization. Do not send the user's entire history to the model by default.

15. API Design

15.1 Example API groups

Group	Example endpoints
Auth	POST /auth/login, POST /auth/logout, POST /auth/refresh
Chat	POST /conversations, POST /conversations/:id/messages, GET /conversations/:id
Health	GET /health/profile, PATCH /health/profile
Memory	GET /health/memories, PATCH /health/memories/:id, DELETE /health/memories/:id
Documents	POST /documents/upload-url, POST /documents/:id/process, GET /documents/:id
Images	POST /images/upload-url, POST /images/:id/analyze
Plans	POST /plans, GET /plans, PATCH /plans/:id
Tasks	POST /tasks, POST /tasks/:id/complete
Reminders	POST /reminders, PATCH /reminders/:id
Health data	POST /health-data/measurements, GET /health-data/trends
HealthKit	POST /integrations/healthkit/sync, DELETE /integrations/healthkit
Care	GET /care/providers, GET /care/specialties

Use typed request/response schemas, versioned APIs, idempotency where appropriate, pagination, rate limits, ownership checks, and structured error codes.

16. Security & Privacy

16.1 Security baseline

- TLS for all network traffic.
- Encryption at rest for database, object storage and backups.
- Keys/secrets in a managed secrets system.
- Never put model API keys in iOS or browser code.
- Strong authentication and optional MFA.
- Short-lived access tokens and refresh-token rotation where appropriate.
- Server-side authorization on every health-data access.
- Separate administrative privileges from normal user access.
- Signed URLs for private files.
- Malware scanning and strict file validation.
- Rate limiting and abuse detection.
- Audit logs for security-sensitive actions.

16.2 Privacy controls

- Explicit consent for health integrations, voice and optional data processing.
- Clear privacy notice explaining what is collected and why.
- User ability to export supported data.
- User ability to correct and delete supported data.
- Retention policies for documents, images, audio and derived data.
- Minimize health content in operational logs.
- Regional data controls where required.
- Vendor/subprocessor inventory and review.

16.3 Threat model

- Account takeover.
- Unauthorized access to health records.
- Cross-user data leakage through retrieval.
- Malicious document/image prompt injection.
- Compromised admin account.
- Stolen API credentials.
- AI-generated unsafe medical guidance.
- Missed urgent escalation.
- AI cost abuse through automated requests.
- Notification leakage on a shared/locked device.

17. UX/UI Specification

17.1 iOS bottom navigation

Tab	Purpose
Home	Daily health snapshot, tasks, trends and AI entry point
Chat	AI health conversation
Health	Profile, measurements, conditions, medications
Plans	Doctor instructions, tasks, reminders
Profile	Account, integrations, privacy and settings

17.2 Home example

```
Good morning
How are you feeling today?

Sleep 7h 12m
Heart rate 72 bpm
Water 3 / 5
Tasks 1 / 2

[ Ask your Health Assistant ]

Today's plan
✓ Morning medication
■ Drink 2 more glasses of water
■ 20-minute walk

Recent health trend
[chart]
```

17.3 Design principles

- Clean and calm visual hierarchy.
- Health-critical warnings must be visually distinct.
- Never bury urgent instructions below decorative UI.
- Make source/provenance visible when showing medical facts.
- Use plain language first; allow deeper technical detail on demand.
- Use progressive disclosure so the home screen remains simple.

18. Monorepo / Codebase

```
health-ai/
■ ■ ■ apps/
■ ■ ■ ■ ios/
■ ■ ■ ■ web/
■ ■ ■ ■ admin/
■ ■ ■ ■ services/
■ ■ ■ ■ api/
■ ■ ■ ■ ai/
■ ■ ■ ■ documents/
■ ■ ■ ■ notifications/
■ ■ ■ ■ health-data/
■ ■ ■ ■ packages/
■ ■ ■ ■ api-types/
■ ■ ■ ■ safety/
■ ■ ■ ■ database/
■ ■ ■ ■ prompts/
■ ■ ■ ■ shared-types/
■ ■ ■ ■ infrastructure/
■ ■ ■ ■ docker/
■ ■ ■ ■ terraform/
■ ■ ■ ■ monitoring/
```

Because the iOS client is Swift, it will live alongside the TypeScript monorepo rather than being forced into a JavaScript mobile framework. Shared API contracts can be generated from OpenAPI so Swift and TypeScript clients remain synchronized.

19. Development Workflow with Claude Code

- Use Claude Code for implementation, refactoring, tests, API generation and documentation, but require human review for security- and health-critical code.
- Maintain a strict repository-level CLAUDE.md with architecture rules, privacy rules, coding conventions and medical safety constraints.
- Implement one vertical slice at a time: auth → chat → memory → safety → documents → plans → HealthKit.
- Require tests for every backend endpoint and safety-critical workflow.
- Never allow AI coding agents to invent clinical rules; clinical rules should come from reviewed specifications.
- Keep secrets out of the repository and development prompts.
- Use pull requests and CI before merging production changes.

Suggested CLAUDE.md rules

- Never expose health data across user boundaries.
- Never add a medication change action without explicit product approval and safety review.
- Never treat AI inference as confirmed medical history.
- All health API routes require authorization checks.
- All uploads require validation and isolated processing.
- All safety behavior must have automated tests.

20. Testing & AI Evaluation

20.1 Software testing

- Unit tests for domain logic.
- Integration tests for database/API flows.
- End-to-end tests for iOS and web critical journeys.
- Upload/security tests.
- Load tests.
- Authentication/authorization tests.
- Backup/restore tests.

20.2 AI evaluation

- Emergency symptom escalation.
- Ambiguous symptoms.
- Incomplete information.
- Medication change requests.
- Document extraction accuracy.
- Conflicting report values.
- Low-quality images.

- Prompt injection inside uploaded documents.
- Cross-user retrieval isolation.
- Hallucinated medical facts.
- Overconfident language.
- Appropriate uncertainty and care escalation.

20.3 Release gate

Health-facing releases should not ship based solely on ordinary software tests. The release gate should include functional testing, security/privacy review, AI evaluation, safety-case review, monitoring readiness, rollback capability and—where appropriate—clinical/specialist review.

21. Operations & Reliability

- Monitor API latency/error rate.
- Monitor AI latency, token usage and cost.
- Monitor document-processing queues.
- Monitor notification delivery.
- Monitor HealthKit integration failures.
- Monitor authentication anomalies.
- Monitor safety escalation events.
- Maintain encrypted backups and test restoration.
- Define RPO/RTO before production.
- Maintain incident response and rollback procedures.

AI failure behavior

If the AI provider fails, the product must clearly indicate that the AI response could not be generated. It must not fabricate an answer or imply that a medical analysis was completed.

22. Compliance & Regulatory Workstream

Health data and AI health functionality can trigger significant privacy and regulatory obligations. Exact requirements depend on the countries served, product claims, data flows, business model, clinical functionality, and whether the product meets a jurisdiction's definition of regulated medical-device software.

- Document intended use and product claims before launch.
- Map privacy and health-data obligations for every target market.
- Review whether any feature could be regulated medical-device functionality.
- Review consent, privacy notice, retention/deletion and data-subject rights.
- Review vendors/subprocessors and data-processing agreements.
- Create incident response and breach procedures.
- Maintain model/provider/version records for production AI systems.
- Obtain qualified legal/regulatory and clinical/safety review before public launch.

23. Roadmap

Phase	Duration	Scope
0 — Foundation	1–2 weeks	PRD, UX, architecture, safety policy, threat model, data model, evaluation plan
1 — MVP	6–10 weeks	iOS + web auth, AI chat, health profile, memory, safety, reports, timeline, plans, reminders
2 — Multimodal	4–6 weeks	Voice, image analysis, improved document extraction
3 — HealthKit	4–8 weeks	HealthKit permissions, ingestion, normalization, trends
4 — Care	3–5 weeks	Provider/specialty/location discovery and care navigation
5 — Advanced	6–12+ weeks	Apple Watch workflows, longitudinal summaries, clinician/family features

Recommended build order

AI Chat → Health Profile → Health Memory → Safety → Medical Reports → Timeline → Doctor Instructions → Tasks/Reminders → Voice → Images → HealthKit → Care Discovery → Advanced workflows.

24. Team & Budget Planning

Role	MVP involvement
Founder/Product	Full-time product decisions, user research, QA
iOS Engineer	Full-time
Backend Engineer	Full-time
AI Engineer	Full-time or strong part-time
UI/UX Designer	Part-time early, more during UX polish
Medical Advisor	Part-time; safety/product review
Security/Privacy	Part-time specialist, especially before launch

Cloud and AI cost will depend heavily on active users, message volume, voice usage, document processing, image analysis, storage and model selection. Build provider abstraction and usage metering from the beginning so expensive workflows can be controlled.

25. Final Product Definition

The final product should feel like a calm, persistent health companion rather than a generic chatbot. The iOS app is the primary personal health surface because it can access Apple HealthKit and native device capabilities. The web application provides the larger-screen experience for conversations, documents, timeline review, trends, account management and administration.

The shared backend is the source of truth for user-authorized health information, while the AI gateway provides controlled access to models. Safety, privacy, provenance and user control are first-class architecture components—not features to add later.

North-star experience

A user should be able to say: “I don’t feel well today,” upload the relevant report or photo, let the system retrieve only the relevant history, receive a cautious explanation and clear next steps, and—when a clinician has already given instructions—have those instructions turned into reminders and measurable tasks. When the situation may be serious, the product should prioritize appropriate professional or emergency care rather than trying to keep the user inside the AI conversation.

End of documentation. This specification is a product and engineering planning document, not medical or legal advice.